

Operating System

Prepared By:

Er. Ayush Chaudhary

Process Scheduling

- When a computer is multi programmed, it frequently has multiple processes competing for the CPU at the same time.
- When more than one process is in the ready state and there is only one CPU available, a choice has to be made which process to run next.
- The part of the operating system that makes the choice is called the scheduler, and the algorithm it uses is called the scheduling algorithm and the mechanism is called scheduling.
- Process scheduling is a mechanism used by the operating system to manage and schedule multiple processes in a system with a single CPU or multiple CPUs.
- The primary goal of process scheduling is to allocate CPU time fairly to all processes to ensure efficient utilization of system resources and maximize system throughput.

Scheduling Criteria: The criteria include the following:

CPU utilization: The CPU should keep as busy as possible. CPU utilization may range from 0 to 100 percent. In a real system, it should range from 40 percent (for a lightly loaded system) to 90 percent (for a heavily used system).

Throughput: If the CPU is busy executing processes, then work is being done. One measure of work is the **number of processes completed per time unit**, called throughput.

Turnaround time: From the point of view of a particular process, the important criterion is how long it takes to execute that process. The **interval** from the time of **submission** of a process to the time of **completion** is the turnaround time. IT is the sum of the periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O.

Turnaround time = waiting time + burst time or

Turn Around time = Exit time (or Completion Time)– Arrival time

Waiting Time: Waiting time is the **sum of the periods spent waiting** in the ready queue.

Waiting time = start time – arrival time or

Waiting time = Turn Around time – Burst time

Response Time: Response time is **the time it takes to start responding**.

Formula:

Response time = Time at which the process gets the CPU for the first time - Arrival time

Burst time: Burst time is a term used in process scheduling to refer to the amount of time it takes for a process to complete its execution on the CPU. It is also known as the **execution time or CPU time**.

Type of Scheduling:

Non-Preemptive Scheduling: In this case, once a process is in the running state, It continues to execute until (a) it terminates or (b) blocks itself to wait for I/O or to request some operating system service.

For Example: First Come First Serve (FCFS), SJF(Shortest Job First), etc.

Preemptive Scheduling: The currently running process may be interrupted and moved to the ready state by the operating system. The decision to preempt may be performed when a new process arrives or periodically based on a clock interrupt OR when a new process switches from the waiting state to the ready state(for example, at the completion of I/O)

For Example: Round Robin, Preemptive SJF, SRTN (Shortest Remaining Time Next), etc.

Batch Scheduling: Batch scheduling is a method of scheduling processes in which **jobs with similar resource** requirements are grouped together into batches, and the batches are then executed in order. Batch scheduling is often used for processing large amounts of data or running repetitive tasks, such as payroll processing or generating reports.

Interactive Scheduling: Interactive scheduling is used in systems where users are **actively engaged** with the computer. This type of scheduling is designed to **minimize**

response time and maximize user productivity. Interactive jobs require immediate attention from the system and are typically short-lived. Examples of interactive tasks include text editing, web browsing, and gaming.

Real-Time Scheduling: Real-time scheduling is used in systems where tasks need to be **executed** within a **specified time frame**. This type of scheduling is used in applications where timing is critical, such as control systems, aviation, and medical devices.

Scheduling Algorithm:

1.FCFS (First Come, First Served):

- FCFS (First-Come, First-Served) scheduling algorithm is a **non-preemptive** scheduling algorithm in which processes are executed based on their arrival time.
- The FCFS algorithm simply schedules the **first process that arrives first and executes it until it is completed** before moving on to the next process in the queue.

Advantage: It is easy to understand and implement.

Disadvantage:

- It may not be very efficient in terms of overall system performance.
- Longer processes may cause shorter processes to wait longer than necessary, leading to poor performance and longer response times.

Consider the following set of processes that arrive at the time 0, with the length of the CPU burst given in milliseconds.



First-Come, First-Served (FCFS) Scheduling

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1, P_2, P_3
The Gantt Chart for the schedule is:



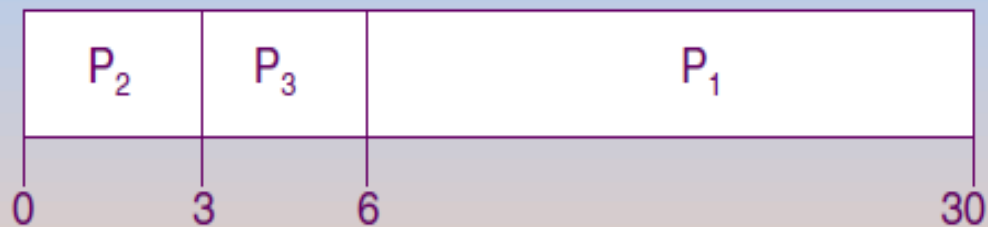
- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order

$$P_2, P_3, P_1.$$

- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case.
- *Convoy effect* short process behind long process

2.SJF(Shortest Job First):

SJF (Shortest Job First) scheduling algorithm is a scheduling policy in which the process with the shortest burst time (the time required to complete a particular task) is executed first.

The SJF algorithm can be preemptive or non-preemptive.

In the **non-preemptive SJF algorithm**, the CPU is allocated to the process with the shortest burst time, and that process continues to run until it completes its task or until a new process with a shorter burst time arrives in the ready queue.

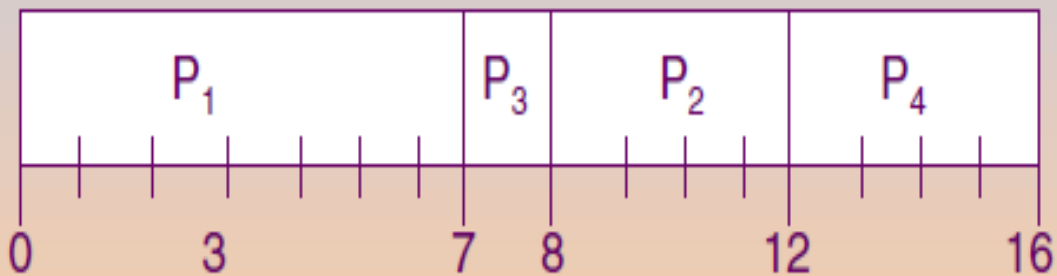
In the **preemptive SJF algorithm**, the CPU can be taken away from the currently running process if a new process with a shorter burst time arrives in the ready queue. In this case, the shorter job is given priority, and the longer job is suspended until the shorter job completes its task.

This is also known as **Shortest-Remaining-Time Next**

Example of Non-Preemptive SJF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

■ SJF (non-preemptive)



■ Average waiting time = $(0 + 6 + 3 + 7)/4 - 4$

3.Shortest-Remaining-Time-Next (SRTN) or Preemptive Shortest Job First

- Shortest-Remaining-Time-Next (SRTN) is a CPU scheduling algorithm used in operating systems.
- It is a preemptive variant of the Shortest-Job-First (SJF) scheduling algorithm.
- In SRTN, the process with the shortest remaining execution time is scheduled next to run on the CPU.
- The algorithm constantly checks for the process with the shortest remaining execution time and preempts the currently running process if a shorter job arrives.
- This ensures that the process with the shortest remaining time is always given the CPU, leading to optimal average turnaround time.

Advantage:

- Less waiting time.
- Quite good response for short processes.

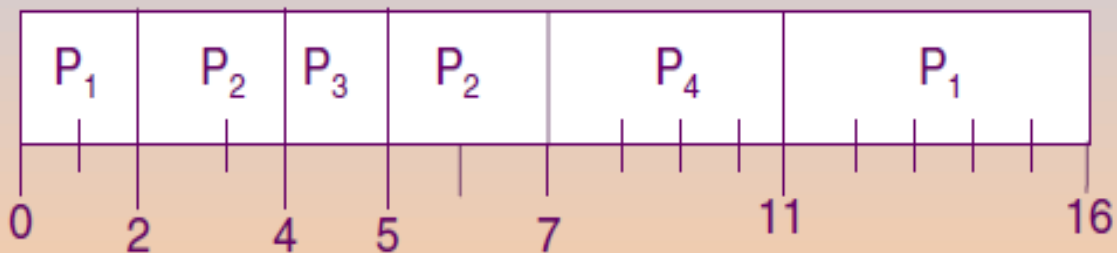
Disadvantage:

- Starvation is possible for long process. Long process may wait forever.
- Context switch overhead is there.

Example of Preemptive SJF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

■ SJF (preemptive)



■ Average waiting time = $(9 + 1 + 0 + 2)/4 - 3$

4. Priority Scheduling

A priority number (integer) is associated with each process. The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority).

◆Preemptive

◆nonpreemptive

SJF is a priority scheduling where priority is the predicted next CPU burst time.

Problem $\equiv$ Starvation –low priority processes may never execute.

Solution $\equiv$ Aging –as time progresses increase the priority of the process.

Consider following set of process arrived at time 0

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P1	10	3
P2	1	1
P3	2	4
P4	5	2

P2	P4	P1	P3
----	----	----	----

0	1	6	16	18
---	---	---	----	----

Average Waiting Time=8.2 ms

Advantage:

- Priority is considered so critical process can get even better response time.

Disadvantage:

- Starvation is possible for low priority processes. It can be overcome by using technique called 'Aging'.
- Aging: gradually increases the priority of processes that wait in the system for a long time.

5.Round Robin Scheduling:

- It is used to allocate processor time to multiple processes in a fair and efficient manner.
- In round-robin scheduling, each process is given a time slice or quantum, and the scheduler assigns the processor to each process in turn for a fixed time period.
- If a process finishes its time slice before completing its task, it is suspended and placed at the end of the queue.
- The scheduler then moves on to the next process in the queue and assigns it the CPU for its time slice.

Advantage:

- Simplest, fairest and most widely used algorithms.

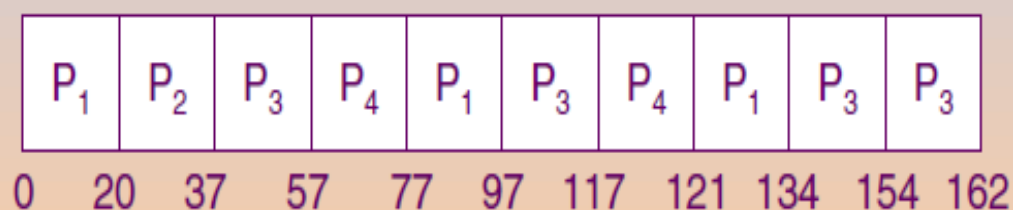
Disadvantage:

- Context switch overhead is there.

Example of RR with Time Quantum = 20

<u>Process</u>	<u>Burst Time</u>
P_1	53
P_2	17
P_3	68
P_4	24

■ The Gantt chart is:



■ Typically, higher average turnaround than SJF, but better *response*.

6. Highest-Response-Ratio-Next(HRN) Scheduling:

- Highest Response Ratio Next (HRNN) is one of the most optimal scheduling algorithms.
- This is a **non-preemptive** algorithm in which, the scheduling is done on the basis of an extra parameter called Response Ratio.
- A Response Ratio is calculated for each of the available jobs and the Job with the highest response ratio is given priority over the others.

Formula:

$$\text{Response Ratio} = \frac{(\text{Waiting Time} + \text{Service Time})}{\text{Service Time}}$$

Advantages:

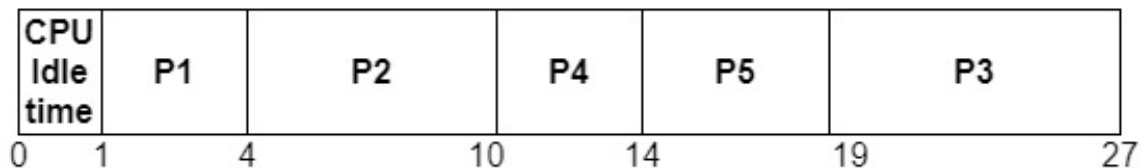
- HRRN Scheduling algorithm generally gives better performance.
- There is a reduction in waiting time for longer jobs and also it encourages shorter jobs.

Disadvantages:

- In this scheduling, there may occur overload on the CPU.

Process	Burst Time	Arrival Time
P1	3	1
P2	6	3
P3	8	5
P4	4	7
P5	5	8

The Gantt Chart according to HRRN will be:



Explanation

Given below is the explanation of the above example

- At time=0 there is no process available in the ready queue, so from 0 to 1 CPU is idle. Thus 0 to 1 is considered as CPU idle time.
- At time=1, only the process P1 is available in the ready queue. So, process P1 executes till its completion.

- After process P1, at time=4 only process P2 arrived, so the process P2 gets executed because the operating system did not have any other option.
- At time=10, the processes P3, P4, and P5 were in the ready queue. So in order to schedule the next process after P2, we need to calculate the response ratio.
- In this step, we are going to calculate the response ratio for P3, P4, and P5.

Response Ratio = $(W+S)/S$

$$RR(P3) = [(10-5) + 8]/8$$

$$= 1.625$$

$$RR(P4) = [(10-7) + 4]/4$$

$$= 1.75$$

$$RR(P5) = [(10-8) + 5]/5$$

$$= 1.4$$

From the above results, it is clear that Process P4 has the Highest Response ratio, so the Process P4 is schedule after P2.

- At time t=10, execute process P4 due to its large value of Response ratio.

- Now in the ready queue, we have two processes P3 and P5, after the execution of P4 let us calculate the response ratio of P3 and P5

$$RR(P3) = [(14-5) + 8]/8$$

$$=2.125$$

$$RR(P5) = [(14-8) + 5]/5$$

$$=2.2$$

From the above results, it is clear that Process P5 has the Highest Response ratio, so the Process P5 is scheduled after P4

- At $t=14$, process P5 is executed.
- After the complete execution of P5, P3 is in the ready queue so at time $t=19$ P3 gets executed.

7. Fair-share CPU scheduling

- The fair-share scheduling algorithm ensures that each process or user receives their fair share of CPU time over time, based on their allocated share.
- This means that even low-priority or background processes will receive CPU time, ensuring that the overall system performance remains responsive and efficient.
- Suppose a computer system has three users, User A, User B, and User C, and a fair-share scheduling algorithm is used to allocate CPU time among them. Each user has been allocated a fair share of CPU time based on their usage history as follows:
 - User A: 30% of CPU time
 - User B: 40% of CPU time
 - User C: 30% of CPU time

Now suppose that User B launches a CPU-intensive application that requires a lot of CPU time to run. In this case, the fair-share scheduler will allocate more CPU time to User B, but will still ensure that Users A and C receive their fair share of CPU time over time.

8. Guaranteed scheduling:

- Guaranteed scheduling is a type of scheduling algorithm used in operating systems to ensure that critical or high-priority processes are guaranteed to receive a minimum level of system resources, such as CPU time.
- In guaranteed scheduling, each process or user is assigned a guaranteed amount of system resources that they are entitled to use.
- This ensures that even in situations where the system is under heavy load or there are many competing processes or users, critical or high-priority processes will receive the resources they need to function properly.
- There are various implementations of guaranteed scheduling in different operating systems.
- For example, in real-time operating systems, a priority-based guaranteed scheduling algorithm may be used to ensure that critical or time-sensitive tasks receive a guaranteed amount of CPU time, with lower-priority tasks receiving less CPU time.

9.lottery scheduling:

In lottery scheduling, each process or user is assigned a number of "lottery tickets" that represent their chance of winning the lottery to receive system resources.

The more lottery tickets a process or user has, the greater their chance of winning the lottery and receiving the resources they need.

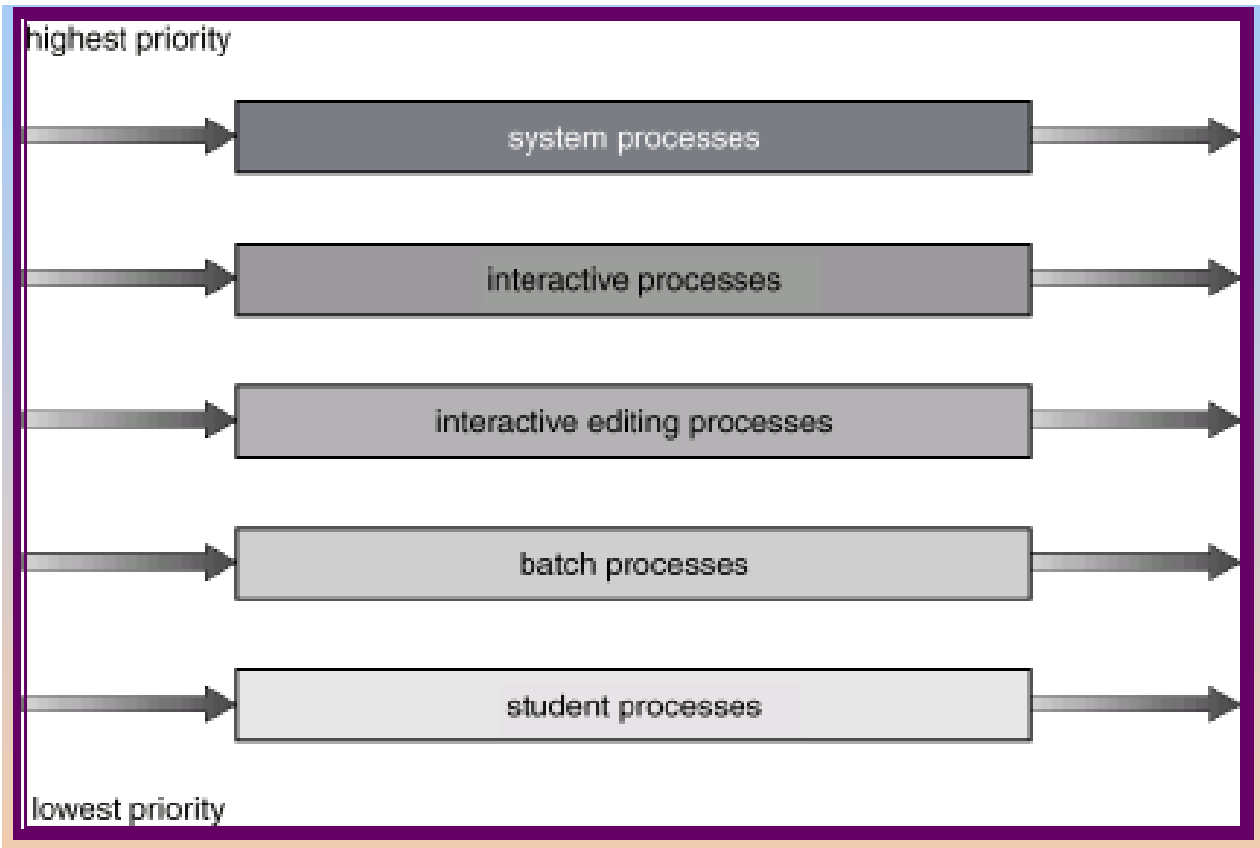
The lottery scheduling algorithm works as follows:

1. Each process or user is assigned a number of lottery tickets based on their resource requirements or priority level.
2. The scheduler randomly selects a lottery ticket from the pool of all lottery tickets.
3. The process or user that holds the winning lottery ticket is allocated the system resources they need.
4. The lottery tickets held by the winning process or user are returned to the pool for the next round of scheduling.

By using lottery scheduling, the system resources are allocated randomly and fairly among all competing processes or users.

10. Multiple Queue Scheduling:

- The ready queue is partitioned into separate queues: foreground (interactive), background (batch)
- Each queue has its own scheduling algorithm,
 - foreground –RR
 - background –FCFS
- Scheduling must be done between the queues.
- Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of **starvation**.
- Time slice –each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR.20% to background in FCFS



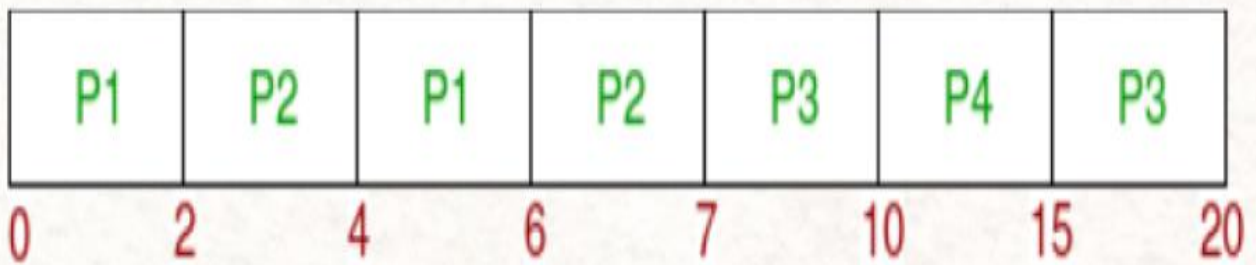
Example:

Consider below table of four processes under multiple queue scheduling. Queue number denotes the queue of the process.

Priority of queue 1 is greater than queue 2.

Queue 1 uses Round Robin (Time Quantum = 2) and queue 2 uses FCFS.

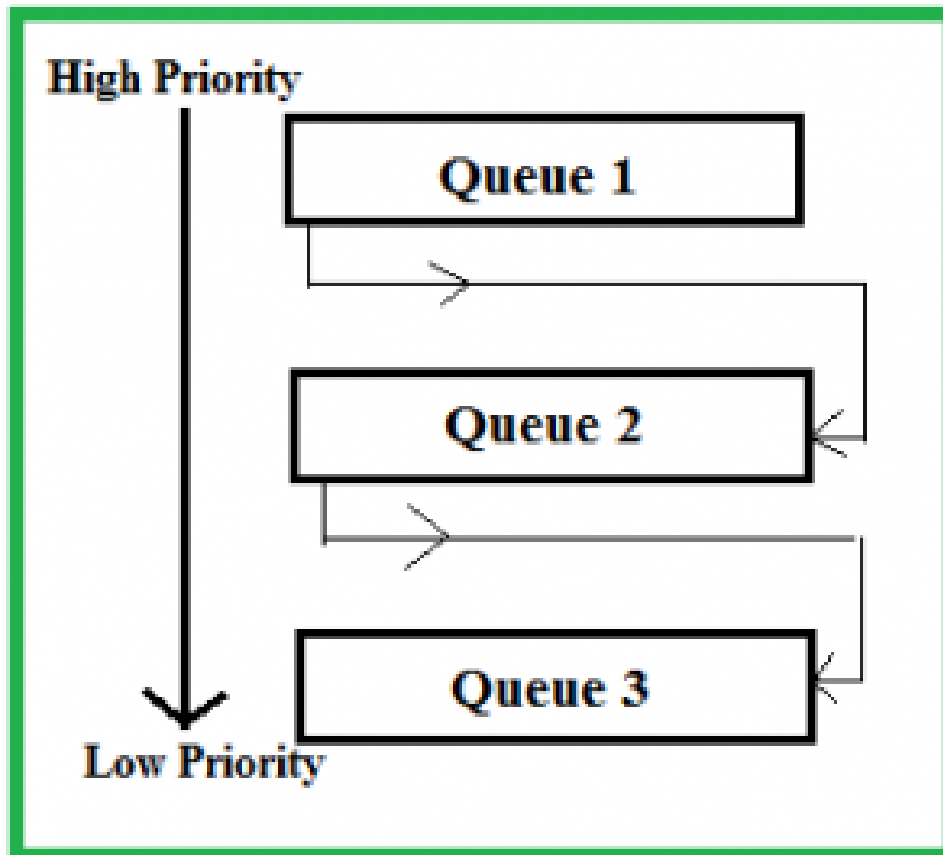
Process	Arrival Time	CPU Burst Time	Queue Number
P1	0	4	1
P2	0	3	1
P3	0	8	2
P4	10	5	1



11. Multilevel Feedback Queue Scheduling:

Multilevel Feedback Queue Scheduling (MLFQ) CPU Scheduling is like Multilevel Queue (MLQ) Scheduling but in this process can move between the queues. And thus, much more efficient than multilevel queue scheduling.

It prevents starvation by moving a process that waits too long for the lower priority queue to the higher priority queue.

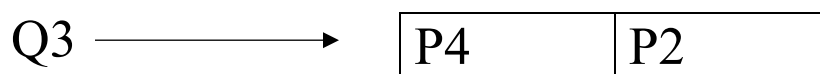
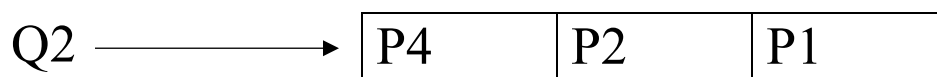
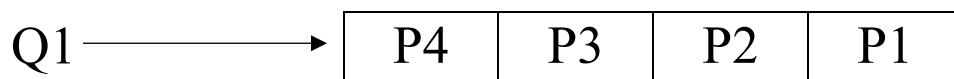


Example:

Consider a multilevel feedback queue scheduling (MLFQ) with three queue Q1, Q2, Q3. Q1 & Q2 use round robin with time quantum (TQ) = 5 & 4 respectively. Q3 use FCFS algorithm. Find Avg. WT & Avg TAT for executing the following process.

Processes	P1	P2	P3	P4
Brust Time	8	22	4	12

	Q1	Q2	Q3
P1	8	3	
P2	22	17	13
P3	4		
P4	12	7	3



Gantt Chart:

P1	P2	P3	P4	P1	P2	P4	P2	P4	
0	5	10	14	19	22	26	30	43	46